CodeRouter AI: Local Model Optimization & Efficiency Report

TECHNICAL ANALYSIS, MODEL CALIBRATION, & FALLBACK ARCHITECTURE PROPOSAL

1. Local Model Analysis: Qwen 2.5 Coder 1.5B

CodeRouter AI currently leverages the local LLM **qwen2.5-coder:1.5b** via Ollama. Despite its small footprint (1.5 billion parameters), this model exhibits remarkable code-generation capabilities owing to its specialized training dataset. To maximize local execution, we must evaluate what it can run reliably versus where it fails.

Category	Supported Local Tasks (High Accuracy)	Unsupported Local Tasks (Requires Remote)
Algorithmic & Math	Prime check, Armstrong numbers, Fibonacci, palindromes, basic sorting (bubble, quicksort), array filtering.	Complex algorithmic optimizations, dynamic programming over large spaces, graph traversal with edge cases.
Syntax Helpers	String manipulations, regular expressions, JSON parsing, dictionary transformations, file I/O operations.	High-level library designs, metadata wrappers, complex reflection or metaprogramming.
Unit Testing & Docs	Generating standard unit tests (PyTest, Jest), docstrings, comments, basic markdown structure.	Comprehensive integration test design, load testing scripts, mock environments for external APIs.
Frameworks & APIs	Basic Flask/FastAPI routes, simple HTML forms, vanilla Javascript handlers, fetch API templates.	Complex database schema design, security middleware, OAuth configurations, multi-service network calls.

2. Maximizing Token Efficiency & Accuracy

To achieve maximum token efficiency while keeping the query accuracy high, we propose a two-fold change:

- **Calibrate Classifier Rules**: Redefine the complexity ratings in classifier.py so that standard algorithmic routines and single-function structures are rated as **Score 2 (Simple)** instead of Score 3. This ensures they fall under the local routing threshold.
- **Increase Complexity Threshold**: Raise the default COMPLEXITY_THRESHOLD to **3** (or 3 by default in configuration). Under this threshold, Complexity 1 (Trivial), 2 (Simple), and 3 (Moderate) queries will be executed locally by default. Only Complexity 4 (Complex) and 5 (Expert) queries will route to the remote Fireworks AI models.

By doing this, **up to 75% of development queries** (which typically consist of function creation, syntax help, and debugging small snippets) can be run locally, yielding **100% token savings** for those queries.

3. Elimination of Fallback Mechanism & Transparent Erroring

Currently, if a task is routed to the remote model (Score > Threshold) and the remote API times out or is missing credentials, the system falls back to running the query locally. While this provides a functional response, it introduces two structural issues:

1. **Degraded Accuracy**: High-complexity tasks (Level 4 and 5) require the reasoning scale of a larger remote model. Running them on a 1.5B local model results in incorrect or severely incomplete code, frustrating the user.
2. **SLA & Latency Overhead**: The user suffers a double latency hit — first waiting 20 seconds for the remote model to timeout, then waiting for the local model to process the complex query, only to receive a sub-par response.

Proposed Behavior:
We will eliminate the local fallback mechanism for remote-bound tasks. If a task is deemed too complex for the local model (Score > Threshold) and the remote API fails or times out, the system will immediately return a clean, descriptive error message:

> *"The task complexity exceeds local model capabilities, and the remote model response timed out. Please check your remote API configuration and try again."*

4. Recommended Code Modifications

- **Modify remote.py (Remove Fallback)**:
  Replace the try-except fallback block in call_remote_model to raise an exception or return a clean error specifying that the remote API call failed. In the workflow graph, if remote_node raises an error, propagate the custom message directly back to the user.
- **Modify router.py & classifier.py (Optimize Threshold)**:
  Update the default COMPLEXITY_THRESHOLD to 3. Calibrate the examples in classifier.py so that standard logic prompts like 'check if number is prime' return a score of 2, ensuring they execute locally.